



MICRO SUN AND SOLUTIONS PLC

Mobile Application Security Testing Requirements Document

Prepared By: Information Network Security Administration (INSA) 2025

Submitted by: Micro Sun and Solutions PLC (MSS)

Submitted to: Information Network Security Administration (INSA)

Table of Contents

1

Introduction

Purpose

Ownership

Scope

Objective of this certificate requested

2

Legal and Administrative Documents

Technical Documentation for Web Application Security Testing

Business Architecture and Design

Features of the Web Application

Security Functionality Document

Secure Coding Standard Documentation

Threat Modeling & Risk Management

Previous Security Testing Reports

OS Supported by the Mobile Application

Hybrid Framework Specifics

Logging & Monitoring Setup

Specific Functionalities for Detailed Testing

Restrictions or Limitations on Testing Approach

Compliance or Regulatory Requirements

Known Vulnerabilities or Security Concerns

3

API Security Audit Requirements

API Types

Authentication Mechanism

Third-Party Integrations

Authorization and Access Control

Full API Documentation

4

Contact Information and Communication Channel

Specific Scope Table

Conclusion

5

Background of Organization

Micro Sun and Solutions PLC (MSS) is an IT company founded in January 1997, specializing in hardware and network solutions. We offer customized solutions tailored to various industries, including government, banking, and telecom. We focus on high-quality software development and implementation, providing comprehensive corporate solutions across multiple sectors. Our



MICRO SUN AND SOLUTIONS PLC

company emphasizes 100% customer satisfaction and aims to deliver quality products and services at an affordable cost.

1. Introduction

1.1 Purpose

This document provides an overview of the Web and Mobile Application Security Testing Requirements Document, compiled for the Information Network Security Administration (INSA). The application in scope is the **Deresegn Point of Sale (POS) and SaaS Application**, which integrates with the Ethiopian Ministry of Revenues (MoR) e-invoicing system.

1.2 Ownership

The owner of this document is the **Deresegn Engineering and Compliance Department**.

1.3 Scope

The scope encompasses the security mechanisms, architectural layouts, and data flows of the Deresegn POS/SaaS application:

- The Flutter-based mobile client interfaces.
- The Laravel 8 (PHP) backend REST API.
- The local SharedPreferences storage handling offline queuing.
- The integration with the Ministry of Revenues (MoR) e-invoicing API.

1.4 Objective of this certificate requested

The objective of requesting this security certificate is to comply with the national cybersecurity mandates established by the Information Network Security Administration (INSA). Obtaining this certification ensures that the Deresegn Point of Sale (POS) and SaaS Application meets the rigorous security, integrity, and confidentiality standards required before public rollout and handling sensitive MoR fiscal data.

2. Legal and Administrative Documents (Mandatory)

The following legal prerequisites and administrative documents are mandatory prior to the finalization of the INSA audit:

1. **Updated Trade License**: [Placeholder for Trade License ID]
2. **TIN Number or National ID**: [Placeholder for TIN Number]
3. **System/Product Patent Certificate (if available)**: [Placeholder for Patent]

3. Technical Documentation for Web Application Security Testing

3.1 Business Architecture and Design

3.1.1 Data Flow Diagram (DFD)



Invoice Initialization: The cashier (User) selects items and initiates an invoice on the Flutter POS Application.

Local Validation: The mobile app validates inventory and necessary fields locally.

Network Check & Queuing:

Offline: The transaction is queued securely in local SharedPreferences.

Online: The request is transmitted via HTTPS to the Laravel backend.

Backend Processing & Authentication: The Laravel API authenticates the user via a JWT token. It checks the specific Branch TIN and fetches the Branch Password to request a temporary MoR token.

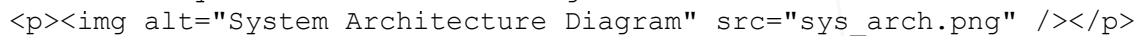


MICRO SUN AND SOLUTIONS PLC

Fiscal Registration: The backend submits the payload to the MoR API. The MoR returns a fiscal receipt number.

Finalization: The backend updates the database and returns a success response.

3.1.2 System Architecture Diagram



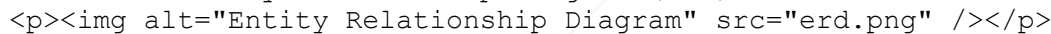
Presentation Layer (Frontend): Flutter (Dart). Employs GetX for state management and routing, and SharedPreferences for offline queuing.

Application Layer (Backend): Built with the Laravel 8 framework.

Data Layer (Backend): Relational MySQL database. Storage for receipts is offloaded to the local server storage.

Integration Layer: Connects securely with the Ministry of Revenues (MoR) API.

3.1.3 Entity Relationship Diagram (ERD)



Key Entities and Definitions:

- **COMPANY & COMPANY_USERS**: `COMPANY` represents the master tenant. `COMPANY_USERS` contains `name`, `phone`, `password`, and `role`.

- **COMPANY_BRANCHES**: Represent the physical stores tied to a company. They contain the critical `tin_number`, `client_id`, and `client_secret` (encrypted via model casts).

- **MOR_INVOICES**: The central fiscal transactional record. Holds `irn`, `document_number`, `status`, and the `full_response` (JSON). Tied to both a Branch and a Company.

- **MOR_RECEIPTS**: Represents actual payments synced with the MoR.

- **SUPPLIES**: Represents inventory items, holding `item_code`, `unit_price`, and `tax_code`.

3.2 Features of the Web Application

Development Frameworks: Flutter (SDK ^3.12.2) and Dart (>=3.0.0) for Frontend; Laravel 8 (PHP) for Backend.

Offline Reliability: Invoices generated during internet outages are converted to JSON and queued locally in SharedPreferences.

Actor Types:

Super Admin / Owner: Global oversight, company registration.

Branch Users: Restricted to specific branch checkout functions.

3.3 Security Functionality Document

3.3.1 Authentication and Access Control



MICRO SUN AND SOLUTIONS PLC

Access Allocation: Users are assigned roles (e.g., owner) upon creation.

API Authentication: Executed via JSON Web Tokens (JWT). The payload is signed with an HS256 cryptographic secret.

3.3.2 User ID Authentication

Authentication utilizes unique Phone Numbers and Passwords tied to specific Company IDs.

3.3.3 Change Password

The application relies on users to set their password during registration or account creation. There is currently no mechanism that forces a user to change their password upon their first login.

3.3.4 User ID / Password Management

Each user registers with a unique Phone Number. The system relies on administrative procedures to discourage the sharing of accounts between cashiers.

3.3.5 Sharing Information

Users are expected to keep their credentials confidential.

3.3.6 Complexity of Password

The backend API enforces a minimum password length of six (6) characters during registration and login. Passwords are mathematically hashed exclusively using Laravel's native Bcrypt implementation

(`Hash::make`). There are no programmatic requirements for special characters, numbers, or mixed casing.

3.3.7 Password Expiry

The application does not currently enforce password expiration or rotation. Passwords remain valid indefinitely unless manually changed by the user or an administrator.

3.3.8 Password History

Password history is not tracked in the database. Users are permitted to reuse old passwords if they choose to change them.

3.3.9 Invalid Login Attempt

If a user submits an incorrect password, the API returns a standard `401 Unauthorized` response. The application currently relies on the hosting provider's infrastructure to handle mass brute-force attacks, as API-level rate limiting (`ThrottlesLogins`) is not explicitly implemented on the authentication endpoints.

3.3.10 Lockout Duration

Because invalid login attempts are not systematically counted in the database, there is no automatic lockout duration enforced on user accounts.

3.3.11 Session Idle Time

The JWT Time-To-Live (TTL) is configured to expire after 60 minutes. Once the 60 minutes elapse, the token is invalidated and the user must re-authenticate.

3.3.12 Encryption & Secure Communication

Data in Transit: The application relies on standard TLS encryption provided by the hosting environment for all API communications.

Data at Rest: Passwords are never stored in plaintext (hashed using Bcrypt). Sensitive MoR credentials (`client_secret`, `api_key`) in the `company_branches` table are



MICRO SUN AND SOLUTIONS PLC

encrypted at rest using Laravel's model casting mechanism

(`encrypted`).

3.4 Secure Coding Standard Documentation

3.4.1 Secure Coding Requirements

Input Validation: API endpoints utilize Laravel Validator logic (e.g., `Validator::make`) to ensure required fields and correct data types are passed.

Database Safety: The Eloquent ORM is utilized to interact with the MySQL database, which utilizes PDO parameterization to mitigate SQL Injection (SQLi) vulnerabilities.

Secrets Management: Database credentials and JWT secrets are injected via environment variables (`.env`).

3.5 Threat Modeling & Risk Management

3.5.1 Asset Identification

Critical (High Risk): MoR fiscal tokens, Branch TIN passwords, user password hashes.

Important (Medium Risk): Invoice historical data.

Public (Low Risk): Item catalogs.

3.5.2 Risk Assessment and Management

Spoofing: Mitigated by stateless JWT validation ensuring requests originate from authenticated users.

Information Disclosure: Prevented by hiding sensitive data (like the `client_secret` and `api_key`) from API JSON responses using Laravel's `$hidden` array on Eloquent models.

3.6 Previous Security Testing Reports

The application relies on standard Laravel and Flutter linting alongside standard functional testing. Any comprehensive penetration testing is performed prior to major production releases locally.

3.7 OS Supported by the Mobile Application

The application supports the Android operating system.

* **Minimum Supported OS**: Android 5.0 (Lollipop) - API Level 21 (Flutter default `minSdkVersion`).

* **Maximum Supported OS**: Android 14 - API Level 34 (Flutter default `targetSdkVersion`).

(Note: iOS is currently out of scope for this specific audit, but the framework is cross-platform capable).

3.8 Hybrid Framework Specifics

The Deresegn mobile application is a hybrid cross-platform application developed using the **Flutter framework (SDK ^3.12.2)** and Dart language.

* **State Management & Routing**: Handled by the `get` plugin (GetX).

* **Network & API Communication**: Managed via the `dio` HTTP client.



MICRO SUN AND SOLUTIONS PLC

* **Storage & Native Interaction:** Uses `flutter_secure_storage` for securely caching tokens, and `shared_preferences` for managing the local offline invoice queue. The application compiles directly to native ARM code, ensuring high performance without relying on web-views for core business logic.

3.9 Logging & Monitoring Setup

The application utilizes the `logger` package to track user activity, network state changes, and security events.

* **Client-Side Logging:** Important application state changes (e.g., `Network restored, attempting to sync offline queue`, `Company authentication required`) and HTTP request failures (e.g., `Retrying GET path after X seconds`) are logged to the console for debugging.

* **Production Alerting:** In production, standard HTTP 4xx and 5xx errors are captured by the backend server logs.

3.10 Specific Functionalities for Detailed Testing

We request that the INSA audit specifically focuses on the following security-critical workflows:

1. **Invoice Generation:** The core flow of generating, validating, and submitting fiscal invoices securely to the MoR.
2. **The Offline-to-Online Sync Mechanism:** Validating the security of invoices temporarily queued in `shared_preferences` during network outages.
3. **MoR Token Exchange:** The backend process of reading the Branch TIN and Password, requesting a temporary MoR token, and signing the fiscal receipt.
4. **Multi-Tenant Authentication:** The JWT-based login flows (`/api/company/login`, `/api/branch/login`) ensuring strict data isolation between companies and branches.

3.11 Restrictions or Limitations on Testing Approach

- **No Denial-of-Service (DoS) or DDoS:** Please refrain from conducting volumetric attacks against the production API (`https://api.deresegn.com`).
- **Safe Testing Environment:** All dynamic testing should be conducted using the provided Test Accounts. Testers must not tamper with or attempt to modify live financial data belonging to other companies in the multi-tenant database.

3.12 Compliance or Regulatory Requirements

The application strictly adheres to the fiscal and invoicing regulations mandated by the **Ethiopian Ministry of Revenues (MoR)**. This requires exact formatting of JSON payloads, strict transmission of tax codes, and secure handling of Branch TINs and passwords.

3.13 Known Vulnerabilities or Security Concerns

- **Password Rotation:** The system currently does not enforce a mandatory password rotation policy or require users to change their passwords upon first login.
- **Brute-Force Protection:** API-level rate limiting (ThrottlesLogins) is currently offloaded to the hosting infrastructure rather than explicitly implemented on the `/api/login` endpoints in the application layer.

```

</ul>
<hr />
<h2>4. API Security Audit Requirements</h2>
<h3>4.1 API Types</h3>
<p>The backend exposes RESTful APIs utilizing HTTP methods and JSON
formats.</p>
<h3>4.2 Authentication Mechanism</h3>
<ol>
<li><strong>Client to Backend</strong>: Authenticated using <strong>JWT (JSON
Web Tokens)</strong> passed in the <code>Authorization: Bearer
<token></code> header.</li>
<li><strong>Backend to MoR</strong>: The Laravel server reads the branch's
specific TIN and Password from the database, exchanges them for a temporary
MoR token, and injects this temporary token into the headers of fiscal
requests.</li>
</ol>
<h3>4.3 Third-Party Integrations</h3>
<p>The primary integration is the <strong>Ethiopian Ministry of Revenues
(MoR) e-invoicing API</strong>. </p>
<h3>4.4 Authorization and Access Control</h3>
<p>Endpoints are protected by JWT middleware to ensure that only
authenticated companies and their respective users can access their financial
records.</p>
<h3>4.5 Full API Documentation</h3>
<p>The mobile client interacts with the backend over HTTPS
(<code>https://api.deresegn.com</code>). </p>
<h4>Core Authentication & Organization Endpoints</h4>
<ul>
<li><strong><code>GET /api/companies</code></strong> </li>
<li><strong>Description:</strong> Retrieves a public list of active companies
for the pre-login picker.</li>
<li>
<p><strong>Response:</strong> Array of objects <code>[{ "id": integer,
"company_name": string }]</code></p>
</li>
<li>
<p><strong><code>POST /api/company/register</code></strong> </p>
</li>
<li><strong>Description:</strong> Registers a new parent company and owner
user.</li>
<li><strong>Request Body:</strong> <code>{ "company_name": string (optional),
"tin_number": string (optional), "owner_name": string (optional), "phone":
string (required), "password": string (required, min: 6) }</code></li>
<li>
<p><strong>Response:</strong> <code>{ "user": object, "token": JWT }</code>
(201 Created)</p>
</li>
<li>
<p><strong><code>POST /api/company/login</code></strong> </p>
</li>
<li><strong>Description:</strong> Authenticates the parent company/owner
user.</li>

```



MICRO SUN AND SOLUTIONS PLC

```

<li><strong>Request Body:</strong> <code>{ "phone": string (required),
"password": string (required), "company_id": integer (optional),
"tin_number": string (optional) }</code></li>
<li>
<p><strong>Response:</strong> <code>{ "user": object, "token": JWT
}</code></p>
</li>
<li>
<p><strong><code>POST /api/branch/login</code></strong> </p>
</li>
<li><strong>Description:</strong> Authenticates a specific branch using its
TIN and branch password.</li>
<li><strong>Request Body:</strong> <code>{ "tin_number": string (required),
"password": string (required) }</code></li>
<li>
<p><strong>Response:</strong> <code>{ "branch": object, "token": JWT
}</code></p>
</li>
<li>
<p><strong><code>POST /api/branches</code></strong> (Requires Bearer
Token)</p>
</li>
<li><strong>Description:</strong> Creates a new branch under the
authenticated company.</li>
<li><strong>Request Body (Multipart Form):</strong> <code>branch_name</code>,
<code>location</code>, <code>phone_number</code>, <code>email</code>,
<code>tin_number</code>, <code>client_id</code>, <code>client_secret</code>,
<code>api_key</code>, <code>password</code>, <code>private_key</code> (file),
<code>certificate</code> (file).</li>
<li><strong>Response:</strong> The created branch object (201 Created)</li>
</ul>
<h4>Supply Catalog Endpoints</h4>
<ul>
<li><strong><code>GET /api/supplies</code></strong></li>
<li><strong>Description:</strong> Retrieves the list of supplies/items in the
catalog.</li>
<li><strong><code>POST /api/supplies</code></strong></li>
<li><strong>Description:</strong> Adds a new item to the supply catalog.</li>
</ul>
<h4>Ministry of Revenues (MoR) Integration Endpoints</h4>
<p><em>All routes below (except <code>/api/refresh/token</code>) require a
<code>Branch-Id</code> header to resolve the specific branch's MoR
credentials.</em></p>
<ul>
<li><strong><code>POST /api/login</code></strong></li>
<li><strong>Description:</strong> Automates the MoR authentication flow using
the branch's securely stored <code>client_secret</code> and
<code>api_key</code>.</li>
<li><strong>Headers:</strong> <code>Branch-Id: &lt;branch_id>&gt;</code></li>
<li>
<p><strong>Response:</strong> Forwards the MoR authorization token.</p>
</li>
</ul>

```




MICRO SUN AND SOLUTIONS PLC

```

<p><strong><code>POST /api/invoice/register</code></strong></p>
</li>
<li><strong>Description:</strong> Registers a new fiscal invoice with the
MoR.</li>
<li><strong>Headers:</strong> <code>Branch-Id: <branch_id></code>,
<code>Mor-Token: <token></code></li>
<li><strong>Request Body:</strong> Follows exact MoR JSON specification for
invoice payload.</li>
<li>
<p><strong>Response:</strong> Forwards MoR response (e.g., success,
validation errors).</p>
</li>
<li>
<p><strong><code>POST /api/cancel/invoice</code></strong></p>
</li>
<li><strong>Description:</strong> Cancels a previously registered MoR
invoice.</li>
<li>
<p><strong>Headers:</strong> <code>Branch-Id: <branch_id></code>,
<code>Mor-Token: <token></code></p>
</li>
<li>
<p><strong><code>POST /api/irn/verfiy</code></strong></p>
</li>
<li><strong>Description:</strong> Verifies an Invoice Reference Number (IRN)
with the MoR.</li>
<li>
<p><strong>Headers:</strong> <code>Branch-Id: <branch_id></code>,
<code>Mor-Token: <token></code></p>
</li>
<li>
<p><strong><code>POST /api/receipt/sales</code></strong> &
<strong><code>POST /api/receipt/withholding</code></strong></p>
</li>
<li><strong>Description:</strong> Submits standard sales and withholding
receipts to MoR.</li>
<li>
<p><strong>Headers:</strong> <code>Branch-Id: <branch_id></code>,
<code>Mor-Token: <token></code></p>
</li>
<li>
<p><strong><code>POST /api/refresh/token</code></strong></p>
</li>
<li><strong>Description:</strong> Refreshes an existing MoR authorization
token.</li>
</ul>
<h4>Invoice Retrieval Endpoints (Read-Only)</h4>
<ul>
<li><strong><code>GET /api/invoices</code></strong> & <strong><code>GET
/api/invoices/{id}</code></strong></li>
<li><strong>Description:</strong> Retrieves historical invoices registered by
the application.</li>
<li><strong><code>GET /api/receipts/{type}/{value}</code></strong> </li>

```



MICRO SUN AND SOLUTIONS PLC

Description: Retrieval endpoint supporting multiple identifier types (e.g., `rrn`, `mor`, `irn`).

5. Contact Information and Communication Channel

Name

Role

Address (Email and Mobile)

Amanuiel Tilahun

CIO

Email: tilahunamanuiel0@gmail.com
Mobile: +251911058179

6. Specific Scope Table

Name of the Assets to be Audited

APK/official link

Test Account

Static Analysis

Dynamic Analysis

Automated Source Code Analysis

Deresegn POS Mobile App

APK Download: <https://github.com/DaV212001/deresegn/releases/latest/download/app-release.apk> GitHub Release v1.0.0
Source Code (Static Analysis): <https://github.com/DaV212001/deresegn> GitHub Repository

Company login:
Phone: 25191134567
Password: password123
Company ID: 9
Branch login:
Tin: 0000037187
Password: Maru@123

Yes

Yes

Yes



MICRO SUN AND SOLUTIONS PLC

```
<td style="text-align: left;">Deresegn Backend API</td>
<td style="text-align: left;">https://api.deresegn.com</td>
<td style="text-align: left;"><em>(Tested via mobile app accounts
above)</em></td>
<td style="text-align: left;">No</td>
<td style="text-align: left;">Yes</td>
<td style="text-align: left;">No</td>
</tr>
</tbody>
</table>
<hr />
```

7. Conclusion

Micro Sun and Solutions PLC is fully committed to maintaining a robust security posture for the Deresegn POS/SaaS application. Through the implementation of strict JWT authentication, secure local storage, local offline queuing, and encrypted MoR token management, we have designed the application to protect sensitive fiscal data at rest and in transit. We welcome INSA's comprehensive security audit to validate our compliance, strengthen our service delivery, and reduce our exposure to cyber threats.